



StockGuard



Módulo 3 · Ejercicio Individual con IA · Dicampus

## Ejercicio Individual de Clase

### StockGuard — QA, Documentación y Pipeline con Python e IA

Módulo 3 · Ejercicio individual incremental · VS Code + GitHub

|                                |                 |
|--------------------------------|-----------------|
| Nombre:                        | Angel Echenique |
| Fecha:                         | 08/04/2026      |
| Enlace GitHub del repositorio: |                 |

**Instrucciones:** Trabaja el proyecto fase a fase, un commit por bloque. Usa la IA con prompts precisos y etiqueta cada uso con `[ai]` en el mensaje del commit. Debes poder explicar en voz alta todo el código de tu repositorio. Revisa el código generado antes de cada push.

#### BLOQUE 1 — Descripción del Proyecto

Trabajarás sobre **StockGuard**, un sistema de gestión de existencias en Python 3.12+ que recibiste de un programador anterior. El código funciona parcialmente pero tiene **errores de lógica, ausencia de tests, documentación inexistente y una vulnerabilidad crítica de seguridad** que permite introducir stock negativo y precios absurdos (sin validación de entrada). Tu misión es auditarlo, documentarlo, testearlo y protegerlo con un pipeline CI/CD.

| Característica  | Detalle                                                                                  |
|-----------------|------------------------------------------------------------------------------------------|
| Lenguaje        | Python 3.12+                                                                             |
| Librerías       | pytest, pytest-mock, flake8 / pylint, dataclasses (stdlib), pyyaml (para GitHub Actions) |
| Commits mínimos | 12 commits con mensajes en convención                                                    |
| Entorno         | VS Code + GitHub (repo público — enlace arriba)                                          |



| Característica    | Detalle                                                   |
|-------------------|-----------------------------------------------------------|
| Duración estimada | 3 – 5 horas de trabajo individual                         |
| IA permitida      | Sí, con criterio. Etiquetar siempre con [ai] en el commit |
| Modalidad         | Individual — cada alumno/a entrega su propio repositorio  |

## BLOQUE 2 — Código de Partida ("legacy code")

Al comenzar el ejercicio el repositorio solo contiene el siguiente archivo `stockguard.py` con código intencionalmente defectuoso:

```
# stockguard.py - código heredado (NO modificar este archivo directamente)

import json, os

INVENTORY_FILE = 'inventory.json'

def load_inventory():
    if not os.path.exists(INVENTORY_FILE):
        return []
    with open(INVENTORY_FILE) as f:
        return json.load(f) # ⚠ sin manejo de JSON corrupto

def save_inventory(items):
    with open(INVENTORY_FILE, 'w') as f:
        json.dump(items, f)

def add_item(name, qty, price):
    items = load_inventory()
    items.append({'name': name, 'qty': qty, 'price': price})
    save_inventory(items) # ⚠ acepta qty y price negativos

def update_price(name, new_price):
    items = load_inventory()
    for item in items:
        if item['name'] == name:
            item['price'] = new_price # ⚠ sin validar new_price
    save_inventory(items)

def get_total_value():
    return sum(i['qty'] * i['price'] for i in load_inventory())
```

**⚠ Vulnerabilidades conocidas:** (1) No se valida que `qty > 0` ni `price > 0`. (2) No hay manejo de archivo JSON corrupto. (3) Ausencia total de tests y documentación. Tu tarea es identificarlas, documentarlas y corregirlas paso a paso.



## BLOQUE 3 — Arquitectura del Proyecto (resultado final)

Esta es la estructura de archivos que debe tener tu repo al entregar:

```
stockguard/
+-- stockguard/
|   +-- __init__.py
|   +-- stockguard.py      # código original (sin modificar)
|   +-- models.py          # dataclass Item: name, qty(>0), price(>0)
|   +-- validator.py       # validate_qty(), validate_price() con docstrings
|   +-- storage.py         # load/save con manejo de archivo corrupto
+-- tests/
|   +-- __init__.py
|   +-- test_validator.py  # tests de validación con pytest
|   +-- test_storage.py    # tests con Mock de sistema de archivos
|   +-- test_models.py     # tests de dataclass Item
+-- .github/
|   +-- workflows/
|       +-- ci.yml        # pipeline: lint + tests en cada push
+-- requirements.txt      # pytest>=8, pytest-mock>=3, flake8>=7
+-- .gitignore            # __pycache__, .venv, *.pyc, inventory.json
+-- README.md             # instalación, uso, sección IA, badge CI
```

## BLOQUE 4 — Plan de 12 Commits — en orden estricto

Cada commit corresponde a una tarea específica. Escribe el mensaje en convención **tipo: descripcion [ai]** cuando la IA ayudó. La columna "Qué debe contener" es el criterio mínimo de aceptación.

### FASE 1 — Estructura y código de partida

| #  | Commit (convención)                        | Qué debe contener                                                                                                                               | Tarea    |
|----|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 01 | chore: init repo<br>y.gitignore            | Repo creado en GitHub, clonado localmente. .gitignore con __pycache__, .venv, *.pyc, inventory.json.                                            | Alumno/a |
| 02 | chore: estructurade<br>carpetas [ai]       | Carpetas stockguard/ y tests/ con __init__.py vacíos. Copiar stockguard.py heredado. requirements.txt con pytest>=8, pytest-mock>=3, flake8>=7. | Alumno/a |
| 03 | docs: auditoria<br>devulnerabilidades [ai] | Archivo AUDIT.md con: lista de las 3 vulnerabilidades identificadas, descripción                                                                | Alumno/a |

| # | Commit (convención) | Qué debe contener                                                                                      | Tarea |
|---|---------------------|--------------------------------------------------------------------------------------------------------|-------|
|   |                     | de cada riesgo y propuesta de corrección. Usa la IA para detectarlas pero redacta las conclusiones tú. |       |

## FASE 2 — Modelo de datos y validación

| #  | Commit (convención)                                  | Qué debe contener                                                                                                                                                                       | Tarea    |
|----|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 04 | feat: dataclass Itemen<br>models.py [ai]             | Clase Item con: name(str), qty(int), price(float). Type hints obligatorios. __post_init__ que lanza ValueError si qty <= 0 o price <= 0. Docstring en formato Google con Args y Raises. | Alumno/a |
| 05 | feat: validator.py –<br>validate_qty y price<br>[ai] | Funciones validate_qty(qty: int) -> bool y validate_price(price: float) -> bool. Docstrings Google completos (Args, Returns, Raises, Example). Integradas en add_item().                | Alumno/a |
| 06 | feat: storage.py –<br>manejo de errores [ai]         | load_inventory() maneja JSONDecodeError y FileNotFoundError devolviendo lista vacía. save_inventory() con indent=2. Docstrings en todas las funciones públicas.                         | Alumno/a |

## FASE 3 — Tests con pytest y Mocks

| #  | Commit (convención)                                     | Qué debe contener                                                                                                                                                                      | Tarea    |
|----|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 07 | test: test_models.py –<br>creacion y validacion<br>[ai] | Tests para: creación válida de Item, ValueError con qty=0, qty=-5, price=0, price=-1. Al menos 1 edge case propio (e.g. price=0.001, qty muy grande). pytest en verde.                 | Alumno/a |
| 08 | test:<br>test_validator.py–<br>casos base [ai]          | Tests para validate_qty y validate_price: valores válidos, límites (0, negativo, muy alto). Usa la IA para generar los casos base y añade 2 edge cases propios.                        | Alumno/a |
| 09 | test: test_storage.py –<br>mock de archivo [ai]         | Usar pytest-mock para simular open() y json.load(). Test de archivo corrupto (JSONDecodeError). Test de archivo inexistente. Test de guardado con indent=2 verificado. Todos en verde. | Alumno/a |



## FASE 4 — Pipeline GitHub Actions

| #  | Commit (convención)                              | Qué debe contener                                                                                                                                                                    | Tarea    |
|----|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 10 | feat:<br>.github/workflows/ci.yml<br>l [ai]      | Pipeline que se ejecuta en push y pull_request a main. Pasos: checkout, setup Python 3.12, pip install, flake8 sobre stockguard/ y tests/, pytest con -v. Badge de estado en README. | Alumno/a |
| 11 | test: verificar<br>pipeline en GitHub<br>Actions | Push al repo. Captura de pantalla del pipeline en verde incluida en README. Corregir cualquier fallo de linting antes del commit.                                                    | Alumno/a |

## FASE 5 — Documentación y cierre

| #  | Commit (convención)                     | Qué debe contener                                                                                                                                            | Tarea    |
|----|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 12 | docs: README final<br>reflexión IA [ai] | README.md con: descripción, instalación, cómo ejecutar tests, sección 'Uso de IA' (qué generó la IA y qué modificaste), captura del CLI y badge CI en verde. | Alumno/a |

## BLOQUE 5 — Formato de Docstrings (obligatorio: estilo Google)

Todas las funciones públicas deben tener docstring en **formato Google**. Ejemplo de referencia para `validate_qty`:

```
def validate_qty(qty: int) -> bool:
    """Valida que la cantidad de un ítem sea un entero positivo.

    Args:
        qty (int): Cantidad a validar.

    Returns:
        bool: True si qty > 0, False en caso contrario.

    Raises:
        TypeError: Si qty no es un entero.

    Example:
        >>> validate_qty(10)
        True
```



StockGuard



```
>>> validate_qty(-1)
False
"""
if not isinstance(qty, int):
    raise TypeError(f"qty debe ser int, recibido {type(qty)}")
return qty > 0
```

**Regla de oro:** Si no tienes docstring, el commit no es válido. Usa la IA para generar el esqueleto y completa los ejemplos tú mismo/a.

## BLOQUE 6 — Estructura del Pipeline (ci.yml)

El archivo `.github/workflows/ci.yml` debe tener al menos estos pasos:

```
name: StockGuard CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Lint with flake8
        run: flake8 stockguard/ tests/ --max-line-length=100

      - name: Run tests with pytest
        run: pytest tests/ -v --tb=short
```

**Importante:** Añade el badge generado por GitHub (Settings → Actions → Badge) al README.md. El pipeline debe estar en verde antes del commit 12.



## BLOQUE 7 — Reglas de Uso de IA

**La IA es tu copiloto, no tu piloto.** Etiqueta con [ai] cada commit donde participó. Debes poder explicar en voz alta todo el código de tu repositorio.

### ✓ Sí puedes hacer

- ✓ Pedir que la IA identifique vulnerabilidades
- ✓ Generar el esqueleto de docstrings y completar ejemplos tú
- ✓ Pedir casos base de tests y añadir edge cases propios
- ✓ Generar el ci.yml y modificar los pasos según tu proyecto
- ✓ Pedir a la IA que explique el código que ha generado

### ✗ NO puedes hacer

- ✗ Copiar código sin leerlo ni entenderlo
- ✗ Pedir 'hazme todos los tests de una vez'
- ✗ No poner [ai] cuando la IA ayudó
- ✗ Aceptar tests sin verificar que pasan en verde
- ✗ No saber explicar el código si el profesor/a pregunta

### Plantilla de prompt recomendada:

```
CONTEXTO: Proyecto StockGuard en Python 3.12, usando pytest y pytest-mock.
ARCHIVO: tests/test_storage.py
NECESITO: Un test que use Mock para simular json.load() lanzando JSONDecodeError
          y verifique que load_inventory() devuelve una lista vacía.
REQUISITOS: usa pytest-mock, añade docstring al test, sigue convención AAA
            (Arrange-Act-Assert).
NO incluyas imports duplicados ni código fuera del test.
```

## BLOQUE 8 — Reflexión Individual

Al terminar el proyecto, responde estas preguntas. **No hay que escribir código, solo demostrar que entendiste.**

## 8.1 — Inventario de uso de IA

Para cada archivo donde usaste IA, describe el prompt principal y qué modificaste respecto al código generado:

| Archivo         | Prompt principal que usaste                                                                                                                                                                                                               | Cambios que hiciste tú                                                                                                                                                                                                                                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| validator.py    | <i>Genera dos funciones, validate_qty y validate_price, con docstring estilo Google, type hints, que lancen TypeError si el tipo no es correcto y devuelvan True si el valor es positivo, False en caso contrario.</i>                    | La IA generó las funciones con los mensajes de error en inglés. Yo los cambié a español para que fueran más claros en el contexto del proyecto. Además, añadí ejemplos específicos en los docstrings y ajusté el manejo de float para que aceptara tanto int como float sin conversiones forzadas.                                                                   |
| storage.py      | <i>Escribe un módulo storage.py con funciones load_inventory y save_inventory. load_inventory debe manejar FileNotFoundError y JSONDecodeError devolviendo lista vacía. save_inventory debe guardar con indent=2. Incluye docstrings.</i> | La IA propuso un código que solo capturaba JSONDecodeError. Yo añadí un bloque except Exception genérico para cualquier otro error (permisos, disco lleno, etc.) y también agregué encoding='utf-8' a la apertura de archivos para evitar problemas con caracteres especiales. Además, completé los ejemplos de los docstrings                                       |
| test_storage.py | <i>Crea tests con pytest-mock para simular json.load() lanzando JSONDecodeError y verificar que load_inventory devuelve lista vacía. También test de archivo inexistente y de guardado con indentación.</i>                               | La IA usó pytest-mock directamente (el decorador mocker). Yo decidí no usar pytest-mock porque preferí unittest.mock.patch para mantener la coherencia con otros tests. Modifiqué todos los tests para usar patch y mock_open. También eliminé la importación de pytest que la IA había incluido pero no se usaba, y añadí un test extra para excepciones genéricas. |
| ci.yml          | <i>Genera un pipeline de GitHub Actions para un proyecto Python 3.12 con pytest y flake8. Debe ejecutarse en push y pull_request sobre main.</i>                                                                                          | La IA generó un pipeline básico con python-version: '3.x'. Yo cambié a la versión exacta 3.12, añadí --max-line-length=100 al comando de flake8 y agregué --statistics para que diera más información. También incluí el paso de actualizar pip antes de instalar dependencias.                                                                                      |

## 8.2 — Análisis de la vulnerabilidad

Explica con tus propias palabras: ¿qué riesgo real tendría permitir qty o price negativos en un sistema de inventario en producción? ¿Cómo lo mitigaste en este ejercicio?

### Riesgo real de permitir qty o price negativos en producción

En un sistema de inventario real, permitir cantidades negativas rompe por completo la lógica de negocio. Imagina un almacén donde se registran ventas y reposiciones. Si un empleado introduce por error -5 unidades, el sistema mostraría un stock negativo, lo que no tiene sentido físico. Eso podría ocultar problemas reales como roturas, robos o errores de conteo. Además, si el sistema se integra con un ERP financiero, el valor total del inventario ( $qty * price$ ) podría volverse negativo, generando informes contables erróneos y potencialmente decisiones de compra o venta equivocadas.

Por otro lado, permitir precios negativos es aún más grave. Un atacante o un usuario malintencionado podría introducir productos con precio -100€, lo que haría que el valor total del inventario disminuyera artificialmente, pudiendo manipular balances o incluso generar órdenes de compra absurdas (el sistema podría "regalar" dinero si se integra con pagos). En el peor de los casos, si el sistema se usa para facturación, un precio negativo podría generar abonos automáticos no deseados.

### Cómo lo mitigué en este ejercicio

- Creé un modelo de datos (Item dataclass) con `__post_init__` que lanza `ValueError` si  $qty \leq 0$  o  $price \leq 0$ . Esto asegura que nunca se pueda instanciar un producto inválido.
- Implementé las funciones `validate_qty` y `validate_price` en `validator.py`, que son llamadas desde `add_item` y `update_price` antes de guardar cualquier cambio.
- En `stockguard.py` (modificado), si la validación falla, se lanza una excepción y no se persiste el dato.
- Los tests unitarios verifican que estas validaciones funcionan correctamente, incluyendo casos borde.

De esta forma, cualquier intento de introducir valores no positivos es rechazado inmediatamente.



### 8.3 — Reflexión sobre el pipeline CI

¿Por qué es útil ejecutar los tests automáticamente en cada push y no solo en local? Razona con una situación concreta de este ejercicio:

Ejecutar los tests en local es necesario durante el desarrollo, pero no es suficiente porque diferentes entornos pueden dar resultados distintos (versiones de Python, dependencias, sistema operativo, etc.). Además, los desarrolladores pueden olvidar ejecutar los tests antes de hacer push.

Un caso concreto ocurrió en este ejercicio: cuando corregí los errores de flake8 localmente, todo pasaba en mi máquina con Windows. Sin embargo, al hacer push, el pipeline en GitHub Actions (Ubuntu Linux) detectó un error que yo no había visto: un `import pytest` no usado en `test_storage.py` que flake8 en Windows no había señalado (aunque debería, pero a veces hay diferencias de versión). El pipeline falló, y eso me obligó a revisar y corregir el código. Sin el pipeline, ese `import` muerto habría llegado al repositorio y podría causar confusión a otro desarrollador o incluso un warning innecesario.

Otro ejemplo: si un compañero de equipo modificara `storage.py` y rompiera el manejo de `JSONDecodeError`, el pipeline lo detectaría inmediatamente al hacer push, sin necesidad de que yo me enterara días después. Así el pipeline actúa como una red de seguridad que garantiza que el código siempre esté en un estado mínimo de calidad (tests pasando y estilo correcto) antes de integrarse.

Además, el pipeline proporciona un **informe público y automático** (el badge verde/rojo) que cualquier miembro del equipo puede consultar, fomentando la transparencia y la disciplina de calidad.



#### 8.4 — ¿Cuándo NO usar IA?

Describe un momento durante el ejercicio en el que decidiste NO usar la IA y lo resolviste tú. ¿Por qué lo hiciste así?

Durante el ejercicio, decidí **no usar la IA para diseñar los casos borde (edge cases) de los tests**. Por ejemplo, en `test_models.py` la IA podía haberme dado casos como `qty=0.001` o `price=10**6`, pero preferí pensar yo mismo en situaciones límite. Razoné que un producto podría tener un precio muy pequeño (0.0001€) en sistemas de microtransacciones, o una cantidad enorme ( $2^{31}-1$ ) para inventarios masivos. También se me ocurrió probar `price=1e-10` para ver si la validación con float funcionaba correctamente.

**¿Por qué lo hice sin IA?** Porque quería entrenar mi propio criterio como QA. Los casos extremos son donde suelen fallar los sistemas, y confiar ciegamente en la IA para generarlos me habría privado de aprender a pensar en condiciones límite.

Otro momento fue al corregir la estructura anidada de carpetas. La IA me habría dado el comando `Move-Item` probablemente, pero preferí entender por qué ocurría el error y resolverlo manualmente mirando el tree /F. Eso me ayudó a comprender cómo Python interpreta los paquetes y la importancia de `__init__.py`. Usar IA habría sido más rápido, pero menos formativo.

En resumen, **no uso IA cuando quiero consolidar un concepto fundamental** o cuando el ejercicio valora específicamente mi capacidad de análisis y creatividad. La IA es una herramienta, no un sustituto del pensamiento crítico..



## BLOQUE 9 — Retos Opcionales — para nota extra

Completa alguno de estos retos si terminas antes de tiempo:

| Reto                       | Descripción                                                                                                                                                                   |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ★ <i>NumPy Docstrings</i>  | Convierte todos los docstrings al formato NumPy y documenta las diferencias con Google en un apartado del README.                                                             |
| ★ <i>Patrón Repository</i> | Refactoriza storage.py a una clase InventoryRepository con get_all(), save_one(), delete_one(). Documenta el patrón en el README y actualiza los tests con mocks de la clase. |
| ★ <i>Coverage Report</i>   | Añade pytest-cov al pipeline. El badge de cobertura debe aparecer en el README. Objetivo: $\geq 80$ % de cobertura.                                                           |
| ★★ <i>Pre-commit hooks</i> | Configura pre-commit con flake8 y black. El README debe explicar cómo instalarlo. Los commits son rechazados si el linter falla.                                              |